BACKEND SECURITY · COHORT NOTES

Know the attack. Block it on your server.

Eight attacks that hit real Express + MongoDB servers. For each one you get what it is in plain words, an animated flow you can step through, the bug in code, and the fix.

Express 5 | MongoDB + Mongoose | express-validator | helmet | express-rate-limit | csrf-csrf

IN THE USER'S BROWSER

The attack runs in the victim's browser and uses their login.

XSS | CSRF

AT YOUR ROUTES

Bad input or stolen credentials reach your controllers and database.

NoSQL injection | Session hijacking | Brute force

AT THE CONNECTION

The server is kept too busy to answer real users.

DoS / DDoS | Slow request | Slowloris

00 The server we protect

One Express + MongoDB API used in every example below

> THE ONE RULE
>
> Everything inside `req.body`, `req.query`, `req.params`, `req.cookies` and `req.headers` comes from the client. The client can be your React app, Postman, or a script someone wrote to attack you. **Treat every value as untrusted until you have checked it.**

All examples use one small API: users can sign up, log in, write posts and comment. The stack is the one you already use: Express, Mongoose, JWT access token in memory, refresh token in a cookie, bcrypt for passwords. Validation is done with `express-validator`.

TERMINAL COPY

```
npm install express mongoose express-validator helmet express-rate-limit cookie-parser c
```

PROJECT STRUCTURE COPY

```
src/
├── app.js                # express app + global middlewares
├── server.js             # app.listen + connection timeouts
├── middlewares/
│   ├── auth.js           # verifies the access token
│   └── validate.js       # sends express-validator errors back
├── models/
│   ├── user.model.js
│   ├── comment.model.js
│   └── session.model.js  # one document per refresh token
├── utils/
│   └── tokens.js         # issueTokens(), sha256()
└── routes/
    ├── auth.routes.js    # /api/auth/*
    ├── post.routes.js    # /api/posts/*
    └── user.routes.js    # /api/users/*
```

The validate middleware

express-validator rules only *collect* errors. This tiny middleware reads them and stops the request with 400 before your controller runs. Every route in these notes uses it.

SRC/MIDDLEWARES/VALIDATE.JS COPY

```
const { validationResult } = require('express-validator');

function validate(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  next();
}

module.exports = validate;
```

Routes used in these notes

ROUTE	METHOD	ATTACKED IN
/api/posts/:postId/comments	POST	01 XSS
/api/users/me/email	POST	02 CSRF
/api/auth/reset-password	POST	03 NoSQL injection
/api/auth/refresh	POST	04 Session hijacking
/api/auth/login	POST	05 Brute force
/api/posts	GET	06 DoS
any route	ANY	07 Slow request, 08 Slowloris

The path of one request, layer by layer

Each layer below stops a different attack. By the end of these notes you will have added all of them.

- **CDN / WAF**

  Drops floods from thousands of IPs before they reach you. Stops DDoS.

- **Load balancer / NGINX + Node timeouts**
  Closes connections that send data too slowly. Stops slow request and Slowloris.
- **`helmet()`**
  Adds security headers like Content-Security-Policy. Helps against XSS.
- **`express-rate-limit`**
  Caps requests per IP. Stops brute force and simple DoS.
- **`express.json({ limit })` + cookie checks**
  Refuses huge bodies. sameSite cookies and CSRF tokens stop CSRF.
- **`express-validator`**
  Checks type, length and format of every field. Stops NoSQL injection and stored XSS.
- **Controller + Mongoose**
  Runs only on clean, validated data.

01 Cross-Site Scripting (XSS)

User text is shown on the page and the browser runs it as code

> `WHAT IT IS`
>
> Your server stores or returns text that a user typed. If that text contains HTML like a `<script>` tag, and the frontend puts it on the page as HTML, **the browser runs it as JavaScript**.
>
> That script runs inside *your* site, with the logged-in user's access. It can read the page, call your API as that user, and send data out.

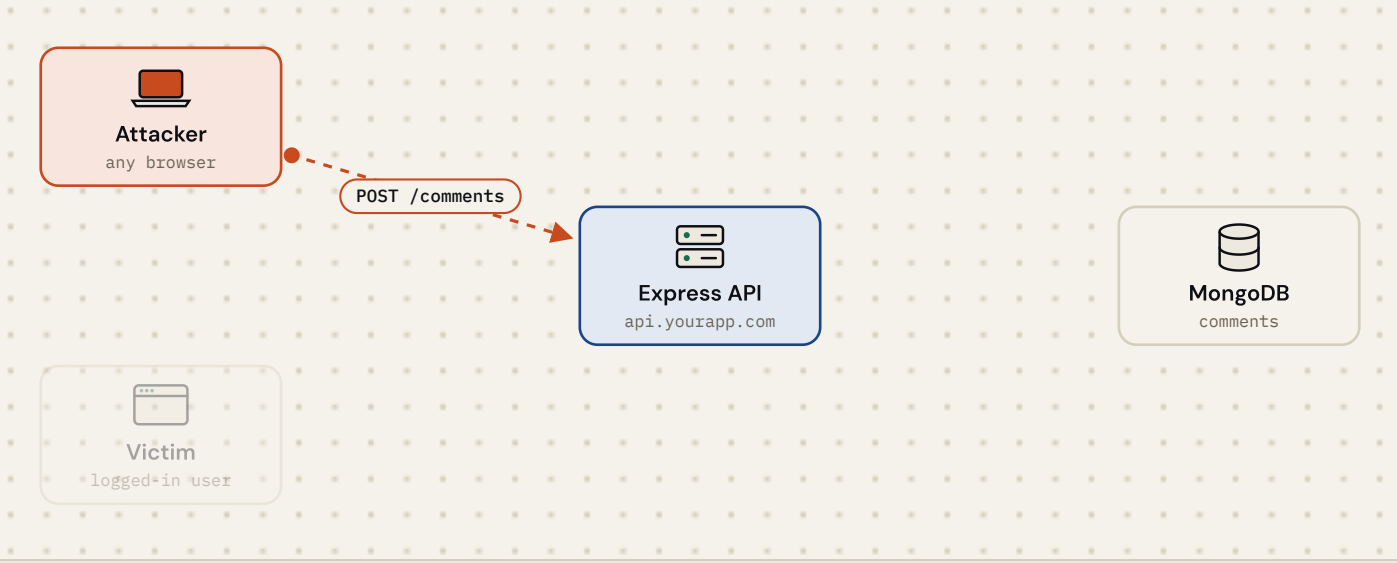
Three kinds of XSS

Type	Where the script lives	Common spot
Stored	Saved in your database, shown to every visitor	Comments, bios, post titles
Reflected	Sent in the request, printed back in the response	A page that prints `?search=` back

Type	Where the script lives	Common spot
DOM-based	Frontend JS puts URL or input into `innerHTML`	Reading `location.hash` into the page

How stored XSS flows

ATTACK FLOW — STORED XSS



ATTACK **Attacker posts a comment**

The attacker sends a comment through the normal API. Inside the text there is an HTML `<script>` tag. To the server it is just a string in a JSON body.

← Back ● ○ ○ ○ ○ ○ Step 1 / 6 Next →

Where the bug is

Two mistakes work together. The server saves the text as-is, and the frontend renders it as HTML.

SRC/ROUTES/POST.ROUTES.JS — VULNERABLE — COPY

```
router.post('/:postId/comments', auth, async (req, res) => {
  const comment = await Comment.create({
    post: req.params.postId,
    author: req.user.id,
    text: req.body.text,            // saved exactly as the client sent it
```

```
  });
  res.status(201).json(comment);
});
```

COMMENTITEM.JSX (REACT) VULNERABLE COPY

```jsx
function CommentItem({ comment }) {
  // turns the text into real HTML, so a script tag inside becomes code
  return <p dangerouslySetInnerHTML={{ __html: comment.text }} />;
}
```

How to stop it

STEP 1 **Validate and escape the input**

`.escape()` replaces `<`, `>`, `&`, `'` and `"` with HTML entities. After that, a browser shows the characters as plain text instead of running them.

SRC/ROUTES/POST.ROUTES.JS FIXED COPY

```
const { body, param, matchedData } = require('express-validator');
const validate = require('../middlewares/validate');

const commentRules = [
  param('postId').isMongoId().withMessage('Invalid post id'),
  body('text')
    .isString().withMessage('Comment must be text')
    .trim()
    .isLength({ min: 1, max: 500 }).withMessage('Comment must be 1 to 500 characters')
    .escape(),                        // < > & ' " become HTML entities
];

router.post('/:postId/comments', auth, commentRules, validate, async (req, res) => {
  const { postId, text } = matchedData(req);   // only fields that passed the rules
  const comment = await Comment.create({ post: postId, author: req.user.id, text });
  res.status(201).json(comment);
});
```

TRY IT · WHAT .ESCAPE() DOES TO THE TEXT

COMMENT TEXT SENT BY THE CLIENT

```
<b>Great post!</b> & "thanks"
```

SAVED IN MONGODB AFTER .ESCAPE()

```
&lt;b&gt;Great post!&lt;&#x2F;b&gt; &amp; &quot;thanks&quot;
```

WHAT THE USER SEES ON SCREEN

Great post! & "thanks"

STEP 2 Render user text as text in React

COMMENTITEM.JSX (REACT) FIXED COPY

```jsx
function CommentItem({ comment }) {
  return <p>{comment.text}</p>;   // React prints it as text, never as HTML
}
```

💡 **Escape once, at one place.** React already escapes `{comment.text}`. If the server also escaped it, users see `<` on screen. For a React-only frontend you can skip `.escape()` and just never use `dangerouslySetInnerHTML` with user text. Keep `.escape()` when the same data also goes into HTML emails, server-rendered pages or an admin panel that uses `innerHTML`.

STEP 3 Add a Content-Security-Policy with helmet

SRC/APP.JS FIXED COPY

```jsx
const helmet = require('helmet');

app.use(helmet());   // sets Content-Security-Policy, HSTS, X-Content-Type-Options and m
```

helmet's default policy only allows scripts from your own domain and blocks inline event handlers. So even if a script tag slips through, the browser refuses to run it. This is your second wall, not your first.

⚠️ **The CSP header must be on the response that serves your HTML page.** If Express only sends JSON and your React build is served by NGINX or a CDN, set the CSP there too. Also remember: the refresh token in an `httpOnly` cookie can't be read by a script, but an injected script can still call your API as the user while the tab is open. Fixing XSS itself matters most.

QUICK CHECK · XSS

A comment is saved in MongoDB and shown to every visitor of the post. Which type of XSS is this?

A — Stored XSS

B — Reflected XSS

QUICK CHECK · XSS

Your refresh token is in an `httpOnly` cookie. Can injected JavaScript read it with `document.cookie`?

A — Yes, any script on the page can read every cookie

B — No, httpOnly cookies are hidden from JavaScript

02 Cross-Site Request Forgery (CSRF)

Another site makes the victim's browser send a request to your API

WHAT IT IS

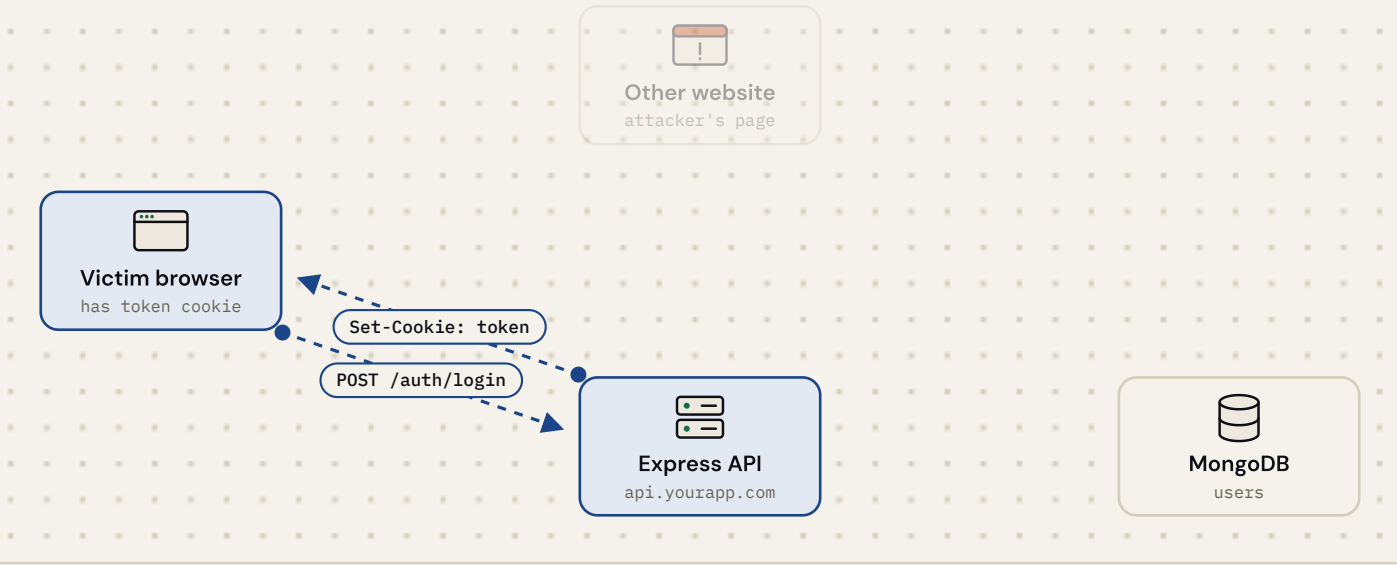
A browser attaches your site's cookies to **every request sent to your domain**, even when that request was started by a page on a different website.

CSRF uses this. A page the attacker controls makes the victim's browser send a request to your API. The cookie goes along automatically, so your server thinks the user did it.

💡 **CSRF only works with cookies.** If your access token lives in memory and is sent in the `Authorization` header, another site cannot add that header, so those routes are safe from CSRF. But many projects keep the JWT in a cookie with `res.cookie('token', ...)`, and your refresh token is always a cookie. That is where CSRF hits.

How CSRF flows

ATTACK FLOW — CROSS-SITE REQUEST FORGERY



NORMAL **Victim logs in**

The user logs in. The server sends the JWT in a cookie named `token`. From now on the browser adds this cookie to every request that goes to your API, on its own.

← Back ● ○ ○ ○ ○ ○ Step 1 / 6 Next →

Where the bug is

AUTH CONTROLLER + MIDDLEWARE + ROUTE — VULNERABLE — COPY

```
// login controller
res.cookie('token', token);                    // no sameSite, no httpOnly, no secure
```

```
// src/middlewares/auth.js
const token = req.cookies.token;            // any request carrying the cookie is truste

// src/routes/user.routes.js
router.post('/me/email', auth, async (req, res) => {
  await User.findByIdAndUpdate(req.user.id, { email: req.body.email });
  res.json({ message: 'Email updated' });
});
```

How to stop it

STEP 1 Set safe cookie options

LOGIN CONTROLLER — FIXED — COPY

```
res.cookie('token', token, {
  httpOnly: true,               // JavaScript can't read it
  secure: true,                 // only sent over HTTPS
  sameSite: 'strict',           // not sent on requests started by other sites
  maxAge: 15 * 60 * 1000,       // 15 minutes
});
```

OPTION	VALUE USED	WHAT IT DOES
httpOnly	true	Hides the cookie from `document.cookie`. Protects against XSS stealing it.
secure	true	Cookie is only sent over HTTPS, never plain HTTP.
sameSite	'strict'	Browser never sends the cookie on a request started by another site. This is the main CSRF fix.
sameSite	'lax'	Sent when the user clicks a link to your site (GET), not on cross-site POST, PUT or DELETE.
sameSite	'none'	Always sent, from any site. Needs `secure: true`. Avoid for auth cookies.
maxAge	900000	Cookie expires after 15 minutes (value is in milliseconds).

💡 **"Site" means the main domain.** `app.yourapp.com` and `api.yourapp.com` are the same site, so `sameSite: 'strict'` still works when your React app and API sit on different subdomains.

STEP 2 Add a CSRF token with csrf-csrf

For cookie-based auth, add a second check. The server gives the frontend a CSRF token. The frontend sends it back in the `x-csrf-token` header on every POST, PUT, PATCH and DELETE. Another site can't read that token, so it can't send it.

SRC/APP.JS — FIXED — COPY

```
const cookieParser = require('cookie-parser');
const { doubleCsrf } = require('csrf-csrf');

app.use(cookieParser());        // must come before doubleCsrfProtection

const { generateCsrfToken, doubleCsrfProtection } = doubleCsrf({
  getSecret: () => process.env.CSRF_SECRET,
  getSessionIdentifier: (req) => req.cookies.token ?? '',   // ties the CSRF token to th
  cookieName: 'x-csrf-token',   // use '__Host-x-csrf-token' in production (needs HTTPS)
  cookieOptions: { sameSite: 'strict', secure: true },
});

// the frontend calls this once and keeps the token
app.get('/api/csrf-token', (req, res) => {
  res.json({ csrfToken: generateCsrfToken(req, res) });
});

// every route registered after this line needs a valid x-csrf-token header
app.use(doubleCsrfProtection);

app.use('/api/users', require('./routes/user.routes'));
```

FRONTEND · API.JS (AXIOS) — COPY

```
const api = axios.create({ baseURL: 'https://api.yourapp.com', withCredentials: true });
```

```
const { data } = await api.get('/api/csrf-token');
api.defaults.headers.common['x-csrf-token'] = data.csrfToken;   // sent on every request
```

STEP 3 Validate the body, and never change data on GET

SRC/ROUTES/USER.ROUTES.JS **FIXED** COPY

```
const { body } = require('express-validator');

router.post(
  '/me/email',
  auth,
  body('email').isEmail().withMessage('Enter a valid email').normalizeEmail(),
  validate,
  updateEmail
);

// never change data on GET: links and <img> tags can fire GET requests from anywhere
// router.get('/me/delete', auth, deleteAccount);   wrong
```

⚠️ **CORS does not stop CSRF.** CORS only decides whether another site can *read* your response. A form POST from another site is still *sent*, the cookie still goes along, and your server still runs the update. Use sameSite cookies and CSRF tokens.

QUICK CHECK · CSRF

Your API's CORS config allows only your frontend's origin. Does that alone stop a CSRF form post from another site?

A — Yes, CORS blocks the request from reaching the server

B — No, the request is still sent; CORS only blocks reading the response

QUICK CHECK · CSRF

Which cookie option stops the browser from sending the cookie on requests started by other sites?

A — sameSite: 'strict'

B — httpOnly: true

03 NoSQL Injection

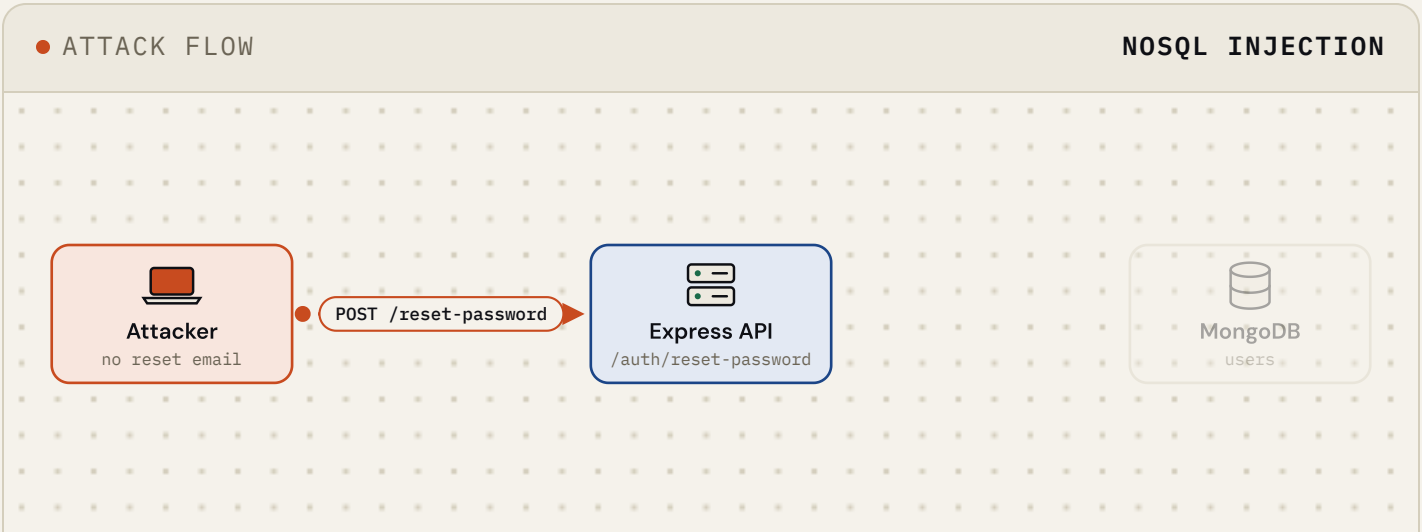
An object arrives where your query expected a string

WHAT IT IS

`express.json()` turns the request body into a JavaScript object. A field you expect to be a string can arrive as **an object** instead.

If that object contains a MongoDB query operator and you pass it straight into `findOne()`, MongoDB reads it as a **condition**, not a value. The meaning of your query changes, for example from "this exact token" to "any token".

How NoSQL injection flows



ATTACK **Attacker calls the reset route**

This route expects `{ token, newPassword }`, where `token` is the random string from a reset email. The attacker never got that email.

Where the bug is

SRC/ROUTES/AUTH.ROUTES.JS VULNERABLE COPY

```
router.post('/reset-password', async (req, res) => {
  const { token, newPassword } = req.body;            // no type check at all

  const user = await User.findOne({
    resetToken: token,                                // could be an object, not a strin
    resetTokenExpiry: { $gt: Date.now() },
  });
  if (!user) return res.status(400).json({ message: 'Invalid or expired link' });

  user.password = await bcrypt.hash(newPassword, 12);
  user.resetToken = undefined;
  await user.save();
  res.json({ message: 'Password updated' });
});
```

How to stop it

STEP 1 **Check the type with express-validator**

The reset token is created with `crypto.randomBytes(32).toString('hex')`, so a real one is always a 64-character hex string. Say exactly that.

SRC/ROUTES/AUTH.ROUTES.JS FIXED COPY

```
const { body } = require('express-validator');

const resetRules = [
  body('token')
    .isString().withMessage('Token must be a string')     // an object fails right here
    .isLength({ min: 64, max: 64 }).withMessage('Invalid token')
    .isHexadecimal().withMessage('Invalid token'),
```

```
  body('newPassword')
    .isString()
    .isStrongPassword().withMessage('Use 8+ characters with upper, lower, number and sym
];

router.post('/reset-password', resetRules, validate, resetPassword);
```

STEP 2 Use only validated data in the controller

SRC/CONTROLLERS/AUTH.CONTROLLER.JS **FIXED** COPY

```
const mongoose = require('mongoose');
const { matchedData } = require('express-validator');

async function resetPassword(req, res) {
  const { token, newPassword } = matchedData(req);    // only fields that passed the rul

  const user = await User.findOne({
    resetToken: token,
    resetTokenExpiry: mongoose.trusted({ $gt: Date.now() }),   // our own operator, see
  });
  if (!user) return res.status(400).json({ message: 'Invalid or expired link' });

  user.password = await bcrypt.hash(newPassword, 12);
  user.resetToken = undefined;
  user.resetTokenExpiry = undefined;
  await user.save();
  res.json({ message: 'Password updated' });
}
```

STEP 3 Turn on Mongoose's safety net

SRC/APP.JS **FIXED** COPY

```
// once, before any query runs
mongoose.set('sanitizeFilter', true);
```

With `sanitizeFilter` on, if an operator object still slips into a filter, Mongoose wraps it so MongoDB compares it as a plain value. Your own operators like `$gt` also get wrapped, so mark them with `mongoose.trusted()` as in step 2.

⚠️ **Never pass the whole `req.body` into Mongoose.** `User.findByIdAndUpdate(id, req.body)` lets a user send any field, including `role: 'admin'`. Pick fields with `matchedData(req)` instead.

💡 **Using Express 5?** The old `express-mongo-sanitize` package throws an error there, because `req.query` became read-only. Express 5's default query parser also no longer builds nested objects from the URL. JSON bodies can still carry objects, which is why the type check in step 1 is your main defense.

QUICK CHECK · NOSQL INJECTION

Which express-validator rule stops an object from reaching your query where a string was expected?

A — body('token').isString()

B — body('token').notEmpty()

QUICK CHECK · NOSQL INJECTION

You turned on `mongoose.set('sanitizeFilter', true)`. Your own query uses `{ $gt: Date.now() }`. What do you do?

A — Wrap it in mongoose.trusted()

B — Nothing, Mongoose knows which operators are yours

04 Session Hijacking

Someone else gets a copy of a valid login token and uses it

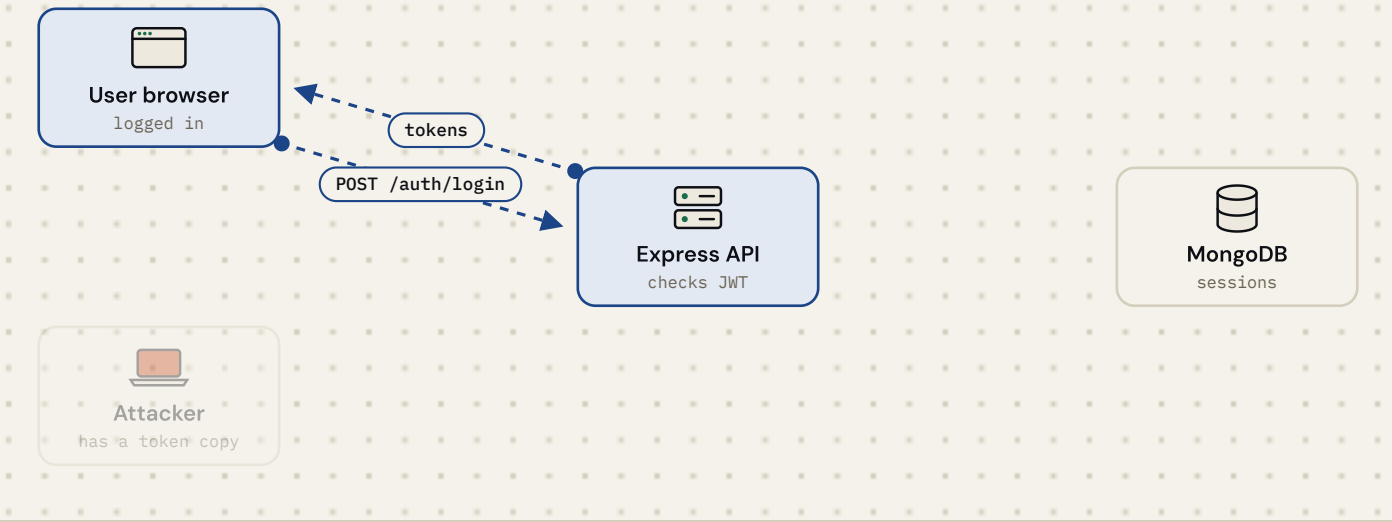
WHAT IT IS

After login, the server gives the browser a token (a JWT, or a session ID). From then on, **whoever sends that token is the user** as far as the server knows.

Session hijacking means an attacker gets a copy of a valid token and sends requests with it. No password needed.

How hijacking flows

ATTACK FLOW — SESSION HIJACKING



NORMAL User logs in

The server returns an access token and sets a refresh token cookie. Whoever holds these tokens is this user, as far as the server knows.

← Back ● ○ ○ ○ ○ Step 1 / 5 Next →

How tokens leak, and the fix for each

Leak	What goes wrong	Fix
XSS	A script reads the token from `localStorage`	Access token in memory, refresh token in an httpOnly cookie, fix XSS (01)

Leak	What goes wrong	Fix
Plain HTTP	Token is readable on the network, e.g. public Wi-Fi	HTTPS everywhere, `secure: true`, HSTS from helmet
Token in URL	`?token=...` lands in server logs and browser history	Send tokens only in headers or cookies
No expiry	A stolen JWT works forever	`expiresIn: '15m'` + refresh token rotation
Fake logout	Logout only clears the frontend, server still accepts the token	Delete the session document in MongoDB

Where the bug is

SRC/ROUTES/AUTH.ROUTES.JS **VULNERABLE** COPY

```
// login
const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);   // never expires
res.json({ token });                                                // frontend saves it

// logout
router.post('/logout', (req, res) => {
  res.json({ message: 'Logged out' });      // the token still works on the server
});
```

How to stop it

STEP 1 A Session model, one document per refresh token

SRC/MODELS/SESSION.MODEL.JS **FIXED** COPY

```
const mongoose = require('mongoose');

const sessionSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true
  tokenHash: { type: String, required: true, unique: true },   // sha256 of the refresh
  usedAt: { type: Date, default: null },                       // set when the token is
```

```
  expiresAt: { type: Date, required: true },
});

// MongoDB deletes the document by itself when expiresAt passes
sessionSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });

module.exports = mongoose.model('Session', sessionSchema);
```

STEP 2 Short access token, hashed refresh token in a strict cookie

SRC/UTILS/TOKENS.JS **FIXED** COPY

```
const crypto = require('crypto');
const jwt = require('jsonwebtoken');
const Session = require('../models/session.model');

const sha256 = (value) => crypto.createHash('sha256').update(value).digest('hex');
const WEEK = 7 * 24 * 60 * 60 * 1000;

async function issueTokens(res, user) {
  const accessToken = jwt.sign(
    { id: user._id, role: user.role },
    process.env.ACCESS_SECRET,
    { expiresIn: '15m' }                    // a stolen copy dies fast
  );

  const refreshToken = crypto.randomBytes(40).toString('hex');   // 80 hex characters
  await Session.create({
    user: user._id,
    tokenHash: sha256(refreshToken),        // store the hash, never the token itself
    expiresAt: new Date(Date.now() + WEEK),
  });

  res.cookie('refreshToken', refreshToken, {
    httpOnly: true,
    secure: true,
    sameSite: 'strict',
    path: '/api/auth',                      // sent only to /api/auth/* routes
    maxAge: WEEK,
  });
```

```
  res.json({ accessToken });                // frontend keeps it in memory, not localSto
}

module.exports = { issueTokens, sha256 };
```

STEP 3 **Rotate on every refresh and catch reused tokens**

Each refresh token works **once**. If an already-used token shows up again, two people hold a copy: the real user and someone else. The server cannot tell who is who, so it ends every session of that user.

SRC/ROUTES/AUTH.ROUTES.JS **FIXED** COPY

```
const { cookie } = require('express-validator');
const { issueTokens, sha256 } = require('../utils/tokens');

router.post(
  '/refresh',
  cookie('refreshToken').isString().isHexadecimal().isLength({ min: 80, max: 80 }),
  validate,
  async (req, res) => {
    const hash = sha256(req.cookies.refreshToken);

    // mark the token as used in one atomic step
    const session = await Session.findOneAndUpdate(
      { tokenHash: hash, usedAt: null },
      { usedAt: new Date() }
    );

    if (!session) {
      // token exists but was already used: a copy is out there
      const reused = await Session.findOne({ tokenHash: hash });
      if (reused) await Session.deleteMany({ user: reused.user });
      res.clearCookie('refreshToken', { path: '/api/auth' });
      return res.status(401).json({ message: 'Please log in again' });
    }

    if (session.expiresAt < new Date()) {
      return res.status(401).json({ message: 'Session expired' });
    }
```

```
    const user = await User.findById(session.user);
    await issueTokens(res, user);            // new access token + new refresh token
  }
);
```

STEP 4 Make logout real

SRC/ROUTES/AUTH.ROUTES.JS **FIXED** COPY

```
router.post('/logout', async (req, res) => {
  const token = req.cookies.refreshToken;
  if (token) await Session.deleteOne({ tokenHash: sha256(token) });
  res.clearCookie('refreshToken', { path: '/api/auth' });
  res.json({ message: 'Logged out' });
});
```

💡 **Using express-session instead of JWT?** Call `req.session.regenerate()` right after a successful login. If you reuse the session ID that existed before login, an attacker who planted that ID can use it (this is called session fixation).

QUICK CHECK · SESSION HIJACKING

With the vulnerable code, the user clicks Logout. Is a stolen copy of the JWT still usable?

A — No, logout makes the token invalid

B — Yes, a JWT stays valid until it expires, and this one never does

QUICK CHECK · SESSION HIJACKING

Why store `sha256(refreshToken)` in MongoDB instead of the token itself?

A — If the database leaks, the hashes can't be used as cookies

B — Hashing makes the cookie smaller to send

05 Brute Force

Guessing passwords by sending login requests again and again

WHAT IT IS

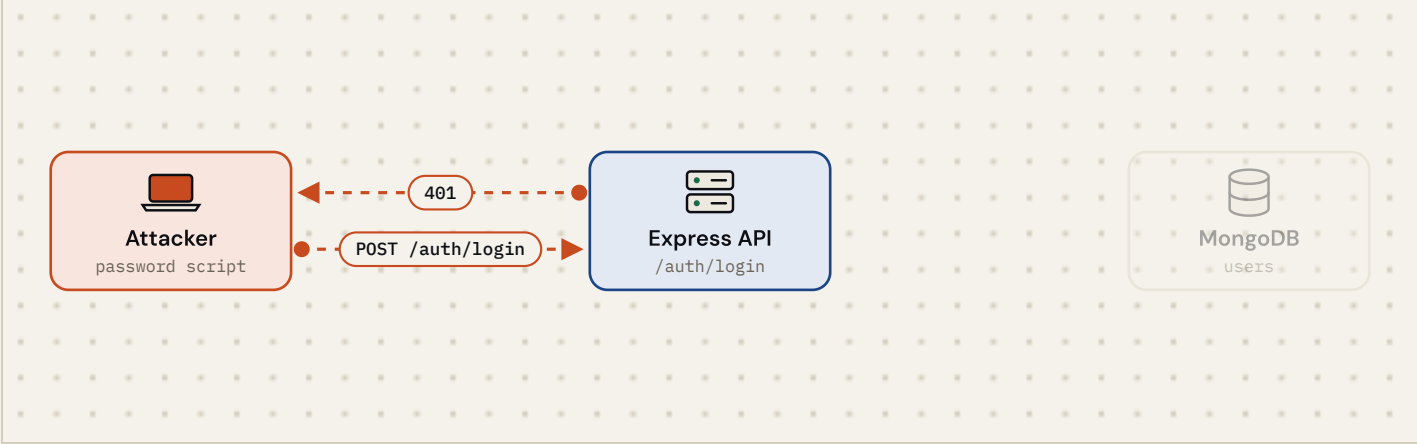
The attacker sends login requests in a loop, each with a different password guess, until one works. A script can send thousands of guesses per minute.

Without limits, your server answers every single one, runs a database query and a bcrypt compare each time, and sooner or later a weak password falls.

Variant	What the attacker tries	What stops it
Brute force	Many passwords against one account	Per-IP rate limit + account lock
Password spraying	One common password against many accounts	Per-IP rate limit + strong password rules
Credential stuffing	Email + password pairs leaked from other websites	Rate limit, account lock, 2FA

How brute force flows

ATTACK FLOW — BRUTE FORCE LOGIN



ATTACK **Guess number 1**

The attacker picks an email and tries a password. Wrong. The server answers 401, and the attacker tries again.

← Back ● ○ ○ ○ ○ Step 1 / 5 Next →

Where the bug is

SRC/ROUTES/AUTH.ROUTES.JS **VULNERABLE** COPY

```
// no limit, and the errors tell the attacker which part was wrong
router.post('/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) return res.status(404).json({ message: 'No account with this email' });

  const ok = await bcrypt.compare(req.body.password, user.password);
  if (!ok) return res.status(401).json({ message: 'Wrong password' });

  await issueTokens(res, user);
});
```

How to stop it

STEP 1 **Validate login and register input**

SRC/ROUTES/AUTH.ROUTES.JS **FIXED** COPY

```
const { body } = require('express-validator');

const loginRules = [
  body('email').isEmail().withMessage('Enter a valid email').normalizeEmail(),
  body('password').isString().isLength({ min: 8, max: 72 }),   // bcrypt only reads 72 b
];

const registerRules = [
  body('email').isEmail().normalizeEmail(),
  body('password')
    .isString()
    .isStrongPassword().withMessage('Use 8+ characters with upper, lower, number and sym
];
```

STEP 2 Rate limit the login route

SRC/ROUTES/AUTH.ROUTES.JS — FIXED — COPY

```
const { rateLimit } = require('express-rate-limit');

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,         // 15 minutes
  limit: 5,                         // 5 failed attempts per IP per window
  skipSuccessfulRequests: true,     // a correct login does not count
  standardHeaders: 'draft-8',
  legacyHeaders: false,
  message: { message: 'Too many login attempts. Try again in 15 minutes.' },
});

router.post('/login', loginLimiter, loginRules, validate, login);
```

OPTION	VALUE USED	WHAT IT DOES
windowMs	900000	Length of the counting window in milliseconds (15 minutes).
limit	5	Requests allowed per IP in one window. Request 6 gets `429 Too Many Requests`.
skipSuccessfulRequests	true	Only failed logins (status 400 and up) are counted.

OPTION	VALUE USED	WHAT IT DOES
standardHeaders	'draft-8'	Sends the standard `RateLimit` headers so clients know when to retry.
legacyHeaders	false	Turns off the old `X-RateLimit-*` headers.
message	{ message }	JSON body sent with the 429 response.

STEP 3 **Lock the account, whatever the IP**

An attacker can switch IPs, so the per-IP limit is not enough on its own. Count failures on the user document too.

SRC/MODELS/USER.MODEL.JS (EXTRA FIELDS) COPY

```
failedLogins: { type: Number, default: 0 },
lockUntil: { type: Date },
```

SRC/CONTROLLERS/AUTH.CONTROLLER.JS FIXED COPY

```
async function login(req, res) {
  const { email, password } = matchedData(req);
  const user = await User.findOne({ email }).select('+password');

  // same answer for "no such email" and "wrong password"
  const invalid = () => res.status(401).json({ message: 'Invalid email or password' });
  if (!user) return invalid();

  if (user.lockUntil && user.lockUntil > Date.now()) {
    return res.status(429).json({ message: 'Too many attempts. Try again later.' });
  }

  const ok = await bcrypt.compare(password, user.password);
  if (!ok) {
    user.failedLogins += 1;
    if (user.failedLogins >= 10) {
      user.lockUntil = new Date(Date.now() + 15 * 60 * 1000);   // lock for 15 minutes
      user.failedLogins = 0;
    }
    await user.save();
    return invalid();
```

```
  }

  user.failedLogins = 0;
  user.lockUntil = undefined;
  await user.save();
  await issueTokens(res, user);
}
```

⚠️ **Behind a load balancer? Set** `app.set('trust proxy', 1)`. Without it, every request looks like it comes from the load balancer's IP, so all your users share one counter and get blocked together. With more than one pod, use a shared store like `rate-limit-redis` so every pod sees the same counts.

QUICK CHECK · BRUTE FORCE

Your API runs behind an AWS load balancer and you forgot `app.set('trust proxy', 1)`. What happens with the login limiter?

A — Every user shares the load balancer's IP, so everyone gets blocked together

B — Nothing changes, the limiter still sees each user's IP

QUICK CHECK · BRUTE FORCE

Why return "Invalid email or password" for both a wrong email and a wrong password?

A — It is shorter to write

B — So the attacker can't find out which emails have accounts

06 DoS & DDoS

Keeping the server so busy that real users can't use it

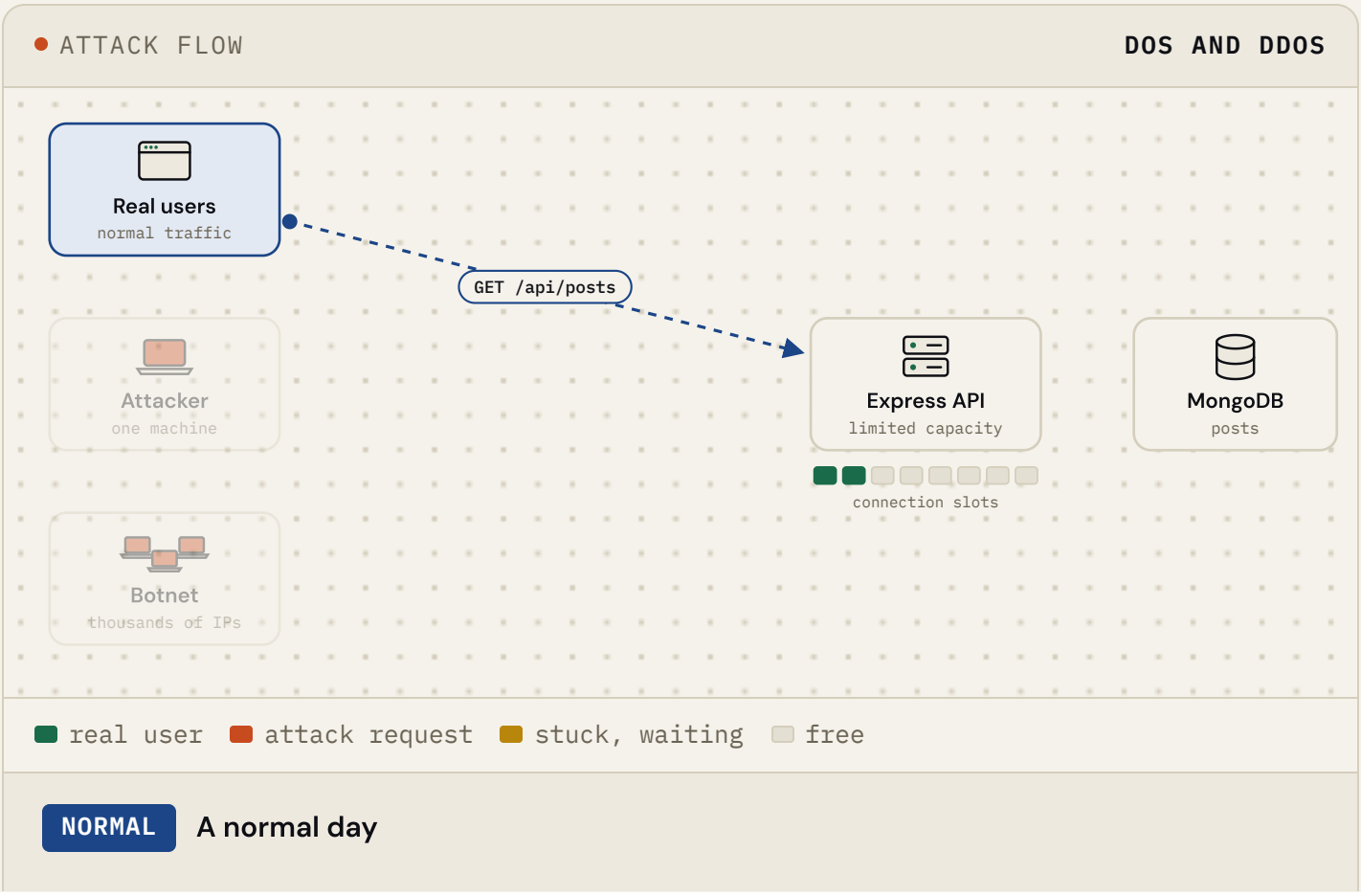
WHAT IT IS

Denial of Service (DoS): make the server so busy that it can't answer real users. Nothing is stolen. The goal is to take the service down.

It comes in two shapes: **too many requests**, or **a few very expensive requests** (a huge JSON body, `?limit=1000000`, a slow query without an index). **DDoS** is the same thing from thousands of machines at once.

What	DoS	DDoS
Source	One machine, one IP	Thousands of hacked machines (a botnet)
Block one IP?	Works ✓	Useless, there are thousands
Per-IP rate limit?	Works ✓	Each IP stays under the limit
Where to stop it	Inside Express	At the edge: CDN, WAF, cloud provider ✓

How DoS and DDoS flow



Your server has a fixed amount of CPU, memory and open connections. Real users use a small part of it.

← Back ● ○ ○ ○ ○ Step 1 / 5 Next →

Where the bug is

SRC/APP.JS + SRC/ROUTES/POST.ROUTES.JS VULNERABLE COPY

```
app.use(express.json());                    // accepts bodies up to the default 100kb,

router.get('/', async (req, res) => {
  const limit = Number(req.query.limit);    // ?limit=1000000 is accepted
  const posts = await Post.find().limit(limit);
  res.json(posts);
});
```

How to stop it

STEP 1 Small body limit

SRC/APP.JS FIXED COPY

```
app.use(express.json({ limit: '10kb' }));                          // bigger bodies get
app.use(express.urlencoded({ extended: false, limit: '10kb' }));
```

STEP 2 A global rate limit for the API

SRC/APP.JS FIXED COPY

```
const { rateLimit } = require('express-rate-limit');

app.set('trust proxy', 1);                  // real client IP from the load balancer

app.use('/api', rateLimit({
  windowMs: 60 * 1000,                      // 1 minute
```

```
  limit: 100,                                  // 100 requests per IP per minute
  standardHeaders: 'draft-8',
  legacyHeaders: false,
}));
```

STEP 3 Cap expensive queries with express-validator

SRC/ROUTES/POST.ROUTES.JS — FIXED — COPY

```
const { query, matchedData } = require('express-validator');

const listRules = [
  query('page').optional().isInt({ min: 1, max: 1000 }).toInt(),
  query('limit').optional().isInt({ min: 1, max: 50 }).toInt(),    // never more than 50
];

router.get('/', listRules, validate, async (req, res) => {
  const { page = 1, limit = 20 } = matchedData(req);

  const posts = await Post.find()
    .sort({ createdAt: -1 })                  // make sure createdAt has an index
    .skip((page - 1) * limit)
    .limit(limit)
    .maxTimeMS(2000)                          // MongoDB stops the query after 2 seconds
    .lean();

  res.json(posts);
});
```

STEP 4 For DDoS, stop it before it reaches Express

No Express middleware can survive traffic from thousands of IPs on its own, because the requests still reach your server and use its network and CPU. Protection has to sit in front of your app.

```
Traffic from 20,000 IPs hits your domain
    ↓
CDN / WAF (Cloudflare, AWS Shield + AWS WAF) drops known-bad and excess traffic
    ↓
Load balancer spreads what is left across pods
```

```
    ↓
Kubernetes HPA adds pods when CPU climbs
    ↓
Express: body limit + rate limit (Redis store) + capped queries ✅
```

QUICK CHECK · DOS & DDOS

A DDoS comes from 20,000 IPs, each sending only 5 requests a minute. Will your per-IP limit of 100 per minute stop it?

A — Yes, the rate limiter blocks it

B — No, every IP stays under the limit; you need edge protection like a WAF or CDN

QUICK CHECK · DOS & DDOS

Which line stops a 50 MB JSON body from being parsed?

A — app.use(express.json({ limit: '10kb' }))

B — app.use(helmet())

07 Slow Request Attack

Sending the request body so slowly that connections stay stuck

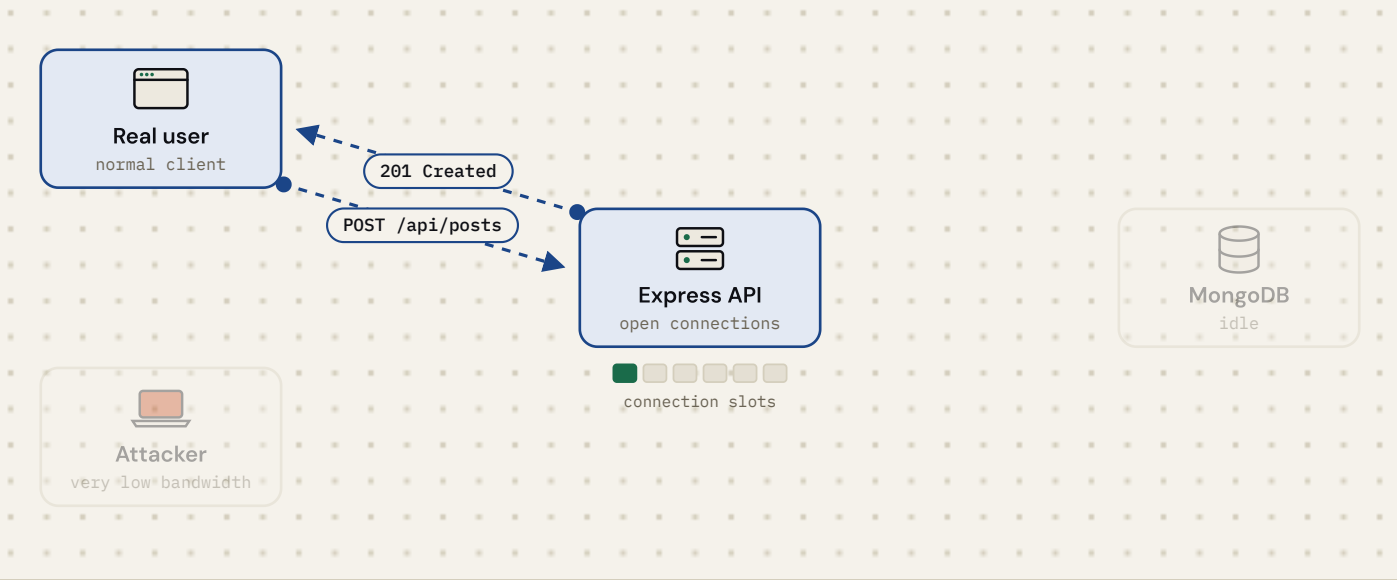
WHAT IT IS

A normal request arrives in milliseconds. In a slow request attack, the client sends valid headers that promise a body, then sends that body **extremely slowly**, just fast enough that the connection never looks dead.

> The server keeps the connection open, waiting for the rest. Repeat this on many connections and the server runs out of room for real users, while almost no traffic shows up on your graphs.

How a slow request attack flows

ATTACK FLOW — SLOW REQUEST (SLOW BODY)



real user · attack request · stuck, waiting · free

NORMAL A normal POST

A normal client sends headers and body in a few milliseconds. Express parses the body, the route runs, and the connection is free again.

← Back · Step 1 / 5 · Next →

> ⚠ **express-validator can't help here.** Validation runs after the whole body has arrived. This attack happens before that, while the body is still coming in. It is stopped at the connection level, with timeouts.

Where the bug is

Since Node 18, `server.requestTimeout` defaults to 5 minutes. That means a connection can be held for up to 5 minutes before Node closes it. Before Node 18 there was no limit at all.

SRC/SERVER.JS — VULNERABLE — COPY

```
const app = require('./app');

app.listen(process.env.PORT || 3000);      // default timeouts: a stuck request may live
```

How to stop it

STEP 1 Set Node's connection timeouts

SRC/SERVER.JS — FIXED — COPY

```
const app = require('./app');

const server = app.listen(process.env.PORT || 3000, () => {
  console.log('API running');
});

server.headersTimeout = 10_000;      // all headers within 10s of the first byte
server.requestTimeout = 30_000;      // headers + body within 30s, or the connection is
server.keepAliveTimeout = 5_000;     // close idle keep-alive connections after 5s
```

SETTING	NODE DEFAULT	WHAT IT DOES
requestTimeout	300000 (5 min)	Deadline for the **whole request**, headers and body. This is the one that stops slow bodies.
headersTimeout	60000 (60 s)	Deadline for the headers only. Stops Slowloris (section 08).
keepAliveTimeout	5000 (5 s)	How long an idle connection stays open after a response.

STEP 2 Keep the body limit from section 06

`express.json({ limit: '10kb' })` rejects a body as soon as it grows past 10 KB, so the attacker can't keep a connection busy with a huge upload either.

STEP 3 Let a reverse proxy face slow clients

In production, NGINX or a load balancer sits in front of Node. It reads the whole request first and only then forwards it to Express. Slow clients get stuck at the proxy, which is built to hold many connections

cheaply.

NGINX.CONF (INSIDE THE SERVER BLOCK) COPY

```
client_header_timeout 10s;   # headers must arrive within 10s
client_body_timeout   10s;   # max wait between two pieces of the body
client_max_body_size  1m;    # refuse bodies above 1 MB
```

💡 **Running behind an AWS ALB?** The ALB faces the slow client, not Node. Then set `server.keepAliveTimeout` higher than the ALB idle timeout (60 s by default), for example `65_000`. If Node closes idle connections before the ALB does, you get random 502 errors.

QUICK CHECK · SLOW REQUEST

Can express-validator stop a slow request attack?

A — Yes, add .isLength() to the body fields

B — No, the body never finishes arriving, so validation never runs

QUICK CHECK · SLOW REQUEST

Which Node setting limits the total time to receive headers *and* body?

A — server.requestTimeout

B — server.keepAliveTimeout

08 Slowloris

Opening many connections whose headers never finish

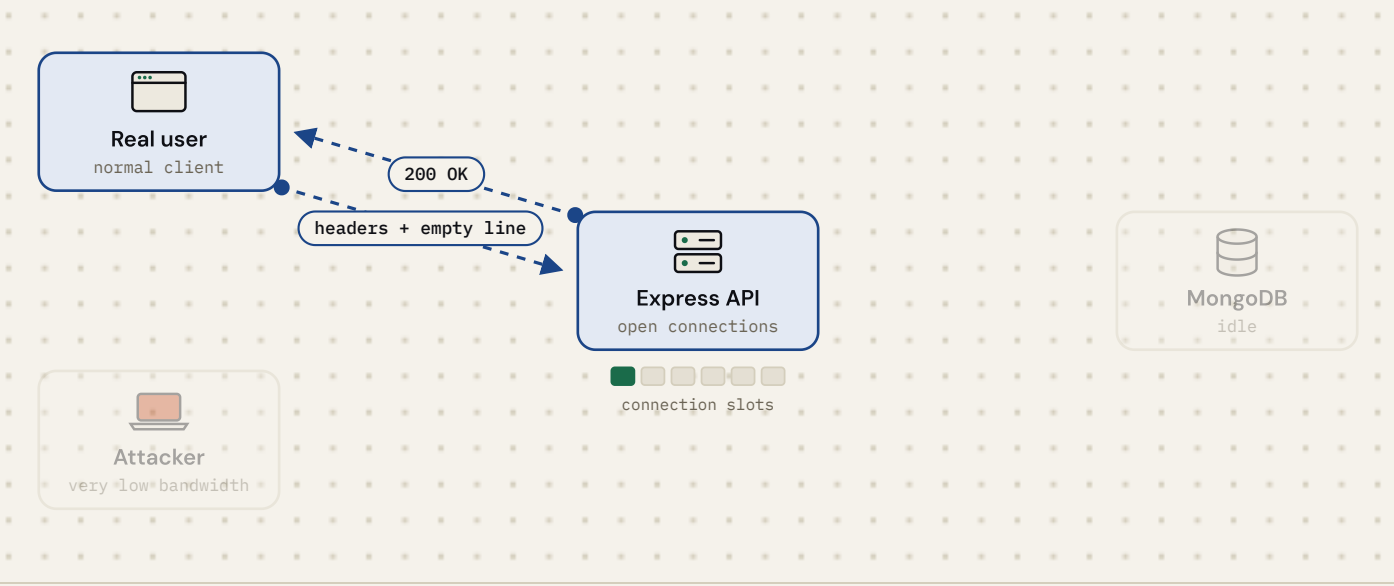
WHAT IT IS

HTTP headers end with **one empty line**. Until that line arrives, the server can't start handling the request, so it keeps waiting.

Slowloris opens many connections and sends only part of the headers on each. Every so often it adds one more header line, so the connection never goes idle. It is a slow request attack aimed at the **headers** instead of the body.

How Slowloris flows

ATTACK FLOW — SLOWLORIS (SLOW HEADERS)



real user · attack request · stuck, waiting · free

NORMAL **A complete request**

HTTP headers end with an empty line. Only after that line does the server know the headers are done and start handling the request.

← Back · Step 1 / 5 · Next →

Flood vs slow body vs Slowloris

What	Flood (DoS)	Slow body (07)	Slowloris (08)
What is slow	Nothing, just a lot of requests	The body	The headers
Bandwidth used	High	Very low	Very low
On your graphs	A clear traffic spike	A few slow uploads	A few slow clients
Stopped by	Rate limit, WAF	requestTimeout, client_body_timeout	headersTimeout, client_header_timeout

> WHY AN IDLE TIMEOUT IS NOT ENOUGH
>
> An idle timeout (`server.timeout`) resets every time a byte arrives. Slowloris sends a byte just before it would fire, so it never fires. `server.headersTimeout` counts the **total** time from the first byte, so the trick stops working.

How to stop it

STEP 1 **Set headersTimeout**

SRC/SERVER.JS — FIXED — COPY

```
server.headersTimeout = 10_000;   // headers not done 10s after the first byte: Node sen
server.requestTimeout = 30_000;   // keep this larger than headersTimeout
```

STEP 2 **Proxy timeout in front**

The same NGINX setting from section 07, `client_header_timeout 10s`, stops Slowloris before it reaches Node. Managed load balancers like AWS ALB also wait for complete headers before they forward anything.

VERIFY **Print the values your server is really using**

SRC/SERVER.JS — COPY

```
console.log({
  headersTimeout: server.headersTimeout,       // expect 10000
  requestTimeout: server.requestTimeout,       // expect 30000
  keepAliveTimeout: server.keepAliveTimeout,   // expect 5000
});
```

💡 **Why old servers suffered most.** Slowloris was famous against servers that gave every connection its own thread, like Apache in prefork mode: a few hundred stuck connections used up every thread. Node does not block a thread per connection, but each open socket still costs memory and a file descriptor, so you still need the timeout.

QUICK CHECK · SLOWLORIS

Why does an idle timeout (`server.timeout`) not stop Slowloris?

A — Each new header line resets the idle timer

B — It only applies to HTTPS connections

QUICK CHECK · SLOWLORIS

Slowloris keeps which part of the request unfinished?

A — The body

B — The headers

09 Defense Checklist

Every attack, what stops it, and the full hardened setup

Attack	First defense	Package / setting
XSS	Validate + escape, render as text, CSP	`.escape()`, no `dangerouslySetInnerHTML`, `helmet()`
CSRF	Cookies not sent cross-site, CSRF token	`sameSite: 'strict'`, `csrf-csrf`
NoSQL injection	Type checks, only validated fields	`.isString()`, `matchedData()`, `sanitizeFilter`
Session hijacking	Short tokens, httpOnly cookie, rotation	`expiresIn: '15m'`, Session model
Brute force	Limit attempts, lock the account	`express-rate-limit`, `failedLogins`
DoS / DDoS	Cheap requests, edge filtering	`express.json({ limit })`, WAF / CDN
Slow request	Deadline for the whole request	`server.requestTimeout`, NGINX
Slowloris	Deadline for the headers	`server.headersTimeout`

Everything together

Order matters. Security headers and rate limits run first so bad traffic is dropped early. Body parsing comes next, then cookies and CSRF, then your routes.

SRC/APP.JS HARDENED COPY

```
const express = require('express');
const helmet = require('helmet');
const mongoose = require('mongoose');
const cookieParser = require('cookie-parser');
const { rateLimit } = require('express-rate-limit');

mongoose.set('sanitizeFilter', true);          // 03 NoSQL injection safety net

const app = express();
app.set('trust proxy', 1);                      // real client IP behind ALB / NGINX

app.use(helmet());                              // 01 XSS: CSP + security headers
```

```
app.use('/api', rateLimit({                    // 06 DoS: 100 req / min / IP
  windowMs: 60 * 1000,
  limit: 100,
  standardHeaders: 'draft-8',
  legacyHeaders: false,
}));
app.use(express.json({ limit: '10kb' }));      // 06 DoS: small bodies only
app.use(cookieParser());

// 02 CSRF: add doubleCsrfProtection here if your auth uses cookies

app.use('/api/auth', require('./routes/auth.routes'));    // 04 + 05: rotation, login li
app.use('/api/posts', require('./routes/post.routes'));   // 01 + 06: escaped comments,
app.use('/api/users', require('./routes/user.routes'));   // 02: validated email change

module.exports = app;
```

SRC/SERVER.JS **HARDENED** COPY

```
const app = require('./app');

const server = app.listen(process.env.PORT || 3000);

server.headersTimeout = 10_000;    // 08 Slowloris
server.requestTimeout = 30_000;    // 07 Slow request
server.keepAliveTimeout = 5_000;   // raise above the ALB idle timeout if you run behind
```